KRONOS: Knowledge-driven Recursive Operations for Network Operations Security - A Self-Morphing AI Framework for Adaptive Cybersecurity

Swayam Singh
School of Computer Science and Engineering
NMIMS University
Mumbai, India
swayam.singh003@nmims.in

Akhil Chauhan
School of Computer Science and Engineering
NMIMS University
Mumbai, India
akhil.chauhan009@nmims.in

Asha Rawat
School of Computer Science and Engineering
NMIMS University
Mumbai, India
asha.rawat@nmims.in

***Abstract*—The speed at which contemporary cybersecurity threats are changing necessitates the use of adaptive defenses that can quickly adjust to new attack trends. This paper presents KRONOS (Knowledge-driven Recursive Operations for Network Operations Security), a self-morphing AI framework composed of three interconnected modules: ORDER (defense), CHAOS (attack simulation), and BALANCE (evolutionary adaptation). To identify potential threats in network traffic, the ORDER module applies anomaly detection methods such as Isolation Forest and related machine learning classifiers. The BALANCE module is designed to support genetic algorithms and reinforcement learning for automated tuning, while the CHAOS module generates simulated attack traffic to continuously stress-test the defense mechanisms. Our implementation demonstrates strong detection performance on the KDD Cup 1999 dataset, achieving high accuracy and low false positive rates in preliminary experiments.**

***Index Terms*—Cybersecurity, Artificial Intelligence, Anomaly Detection, Intrusion Detection Systems, Adversarial Machine Learning, Self Adaptive Systems**

I. Introduction

Threat actors are using increasingly complex attack techniques that put conventional defenses to the test, causing a dramatic change in the cybersecurity landscape in recent years. Because contemporary cyber threats are dynamic, static rule based intrusion detection systems (IDS) and intrusion prevention systems (IPS) are no longer adequate [1]. The advent of polymorphic malware, zero day exploits, and advanced persistent threats (APTs) calls for the creation of adaptive defense systems that can change in real time.

Conventional cybersecurity methods have a number of serious drawbacks. Initially, they depend on preset rules and signatures that are rendered ineffective when attackers alter their strategies. Second, they don't constantly learn from fresh threat intelligence; instead, they function independently. Thirdly, they are unable to test their own efficacy in advance against new attack vectors. The sophistication of contemporary attacks and the capabilities of current defense systems now differ significantly as a result of these flaws.

In order to tackle these issues, this paper presents KRONOS (Knowledge driven Recursive Operations for Network Operations Security), a novel self morphing AI framework with a tri modular architecture. Three complementary parts make up the suggested system: BALANCE (Behavioral Adaptation and Learning through Automated Network Configuration Evolution), CHAOS (Cyber Hardening through Adversarial Operations and Simulation), and ORDER (Operational Real time Defense Enhancement and Response). In order to create a complete adaptive cybersecurity solution, each module performs a unique yet complementary function.

Using cutting edge anomaly detection algorithms like Isolation Forest, One Class SVM, and deep learning based autoencoders, the ORDER module serves as the main defense mechanism. To find possible security risks, it continuously examines system logs, network traffic, and behavioral patterns. By creating complex attack scenarios, the CHAOS module functions as an internal adversary that constantly tests and challenges the defenses. By using a red team approach, the system is protected from changing threat landscapes. The evolutionary adaptation process is coordinated by the BALANCE module, which optimizes detection parameters and architectural configurations based on performance feedback through the use of genetic algorithms and reinforcement learning.

Among our contributions to cybersecurity are: (1) KRONOS, a new tri modular self morphing AI architecture that integrates attack simulation, defense, and evolutionary adaptation; (2) an implementation framework that shows real time adaptation capabilities on benchmark datasets; (3) a thorough performance evaluation that reveals higher detection rates than conventional IDS/IPS systems; and (4) a thorough examination of scalability issues and solutions for practical deployment

scenarios.

II. Background and Related Work

A. Traditional Intrusion Detection Systems

Intrusion detection systems have been recognized as an essential part of network security for over 20 years. Early implementations like Snort [2] relied primarily on signature based detection, matching against known attack patterns in network traffic. They suffer from high false positive rates and are also unmanned to detect new attacks, though they are able to defend against known attacks [3].

Network applications that unite both protocol level inspection with behavioral monitoring, network based intrusion detection systems (NIDS) such as Bro/Zeek [4] provided more sophisticated analysis capabilities. However, these system are non-adaptive and they still rely on predetermined rules. Host-based intrusion detection systems (HIDS) like OSSEC [5],make changes by monitoring system calls, file changes and log analysing.

B. Machine Learning in Cybersecurity

Application of machine learning techniques to cybersecurity has enhanced detection capabilities with promising results. Data mining and machine learning approaches for cyber security intrusion detection are reviewed in detail by Buczak and Guven [6]. Machine learning methods, including artificial neural networks, decision trees and Support Vector Machines (SVM), have been shown to be effective in classification tasks [7].

Anomalous detection techniques are particularly interesting because they can identify attacks that have not been seen on systems before. The Isolation Forest algorithm [8] is particularly suitable for analyzing network traffic due to its excellent performance in finding outliers from high-dimension data. Another popular technique for novelty detection in cyber security applications is One Class SVM [9].

C. Adversarial Machine Learning

Adversarial machine learning is a fast-growing area of research, especially in the domain of cybersecurity applications [10]. Adversarial examples can have substantial effect on machine learning based security solutions, we need to develop effective defense mechanisms [11].

Papernot et al. [12] exposed weaknesses in deep neural networks for malware detection, motivating techniques such as adversarial training. Generative adversarial network (GAN) has been used for attack generation and defense boaster in the filed of cybersecurity [13].

D. Evolutionary Algorithms in Security

Evolutionary algorithms have been widely utilized in a variety of security domains, like tuning IDS parameters to buttress system reliability or vulnerability assessment with automation [14]. Genetic algorithms are used for intrusion detection (as the famous quote goes: ”an expert, in this case, is toothless”), and security policy optimization [15], [16]; moreover, we can find dozens of other examples. Reinforcement learning models have been posed as the basis for adaptive systems that are truly cyber secure, and also intelligent [17]. Such designs can enable continuous self learning and policy optimisation based on the environment feedback keys issued. When combined with machine learning techniques, evolutionary algorithms provide an excellent framework for building systems that can adapt on their own.

III. System Architecture

The Self Morphing AI framework employs a tri modular architecture designed for maximum adaptability and resilience. Figure 1 illustrates the overall system design and information flow between components.

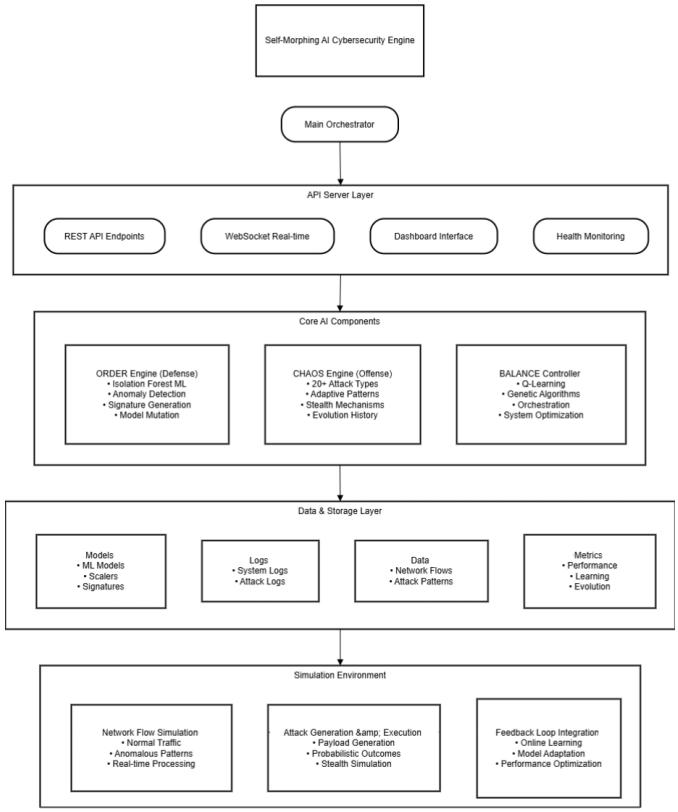

Fig. 1. Overall architecture of the self-morphing AI framework

A. ORDER Module: Operational Real-time Defense Enhancement and Response

The framework’s main defensive element, the ORDER module, implements several tiers of threat detection and response systems. Each layer of its hierarchical architecture offers unique detection capabilities tailored to various threat categories.

The preprocessing layer manages the extraction of features and the intake of raw network traffic. High performance libraries are used to collect network packets, which are then

subjected to a number of filters to extract pertinent data such as statistical metrics, payload characteristics, packet headers, and temporal patterns. To produce traffic representations, feature extraction procedure uses both conventional statistical techniques and deep learning based autoencoders.

To optimize detection coverage, the anomaly detection layer employs a number of parallel techniques. Multiple decision trees with different contamination parameters are maintained by the core algorithm, which is an efficient implementation of Isolation Forest. While keeping a high sensitivity to new attack patterns, our ensemble technique lowers false positives. LSTM neural networks for temporal pattern analysis and One Class SVM for boundary-based anomaly detection are examples of supplemental detection algorithms.

A multi-stage classification pipeline is used by the threat classification component. Deep neural network analysis is used for advanced assault identification after the initial classification, which uses lightweight machine learning models for quick threat categorization. For various attack types, such as DDoS, port scanning, malware communication, and data exfiltration attempts, the system keeps distinct models.

Figure 2 depicts the internal architecture and data flow within the ORDER module.

B. CHAOS Module: Cyber Hardening through Adversarial Operations and Simulation

The CHAOS module acts as an insider attacker and provides the order system with attack scenarios which are used to train or even upgrade defensive assets of the ORDER model. This 'red team' methodology ensures that the system remains resilient against emerging threat environments, through repeated practice and 'weakness' identification.

The attack generation platform uses a variety of techniques for generating realistic threat situations. Rule based generators produces attacks under known attack patterns and signatures, while machine learning based generator leverages GANs trained on real world attack data to generate new types of attacks. The system keeps separate overflows for different types of attack, with some reliance for performance on the kind of threat and OS target systems.

Adversarial training is enriched with advanced methods of generating adversarial examples toward the machine-learning parts of ORDER module. This includes gradient-based attacks like Fast Gradient Sign Method (FGSM) and Projected Gradient Descent (PGD), as well as evolutionary attacks that use Genetic Algorithms to evolve attack parameters in order to optimally penetrate defense configurations.

The simulated environment is used to test attacks and defense effectiveness. It provides pragmatic virtual network topologies of simulated production environments so that you can address a virtually real attack simulation and stay away from affecting your organization's own operational systems. Both automated testing cases and manual red teams can use the environment.

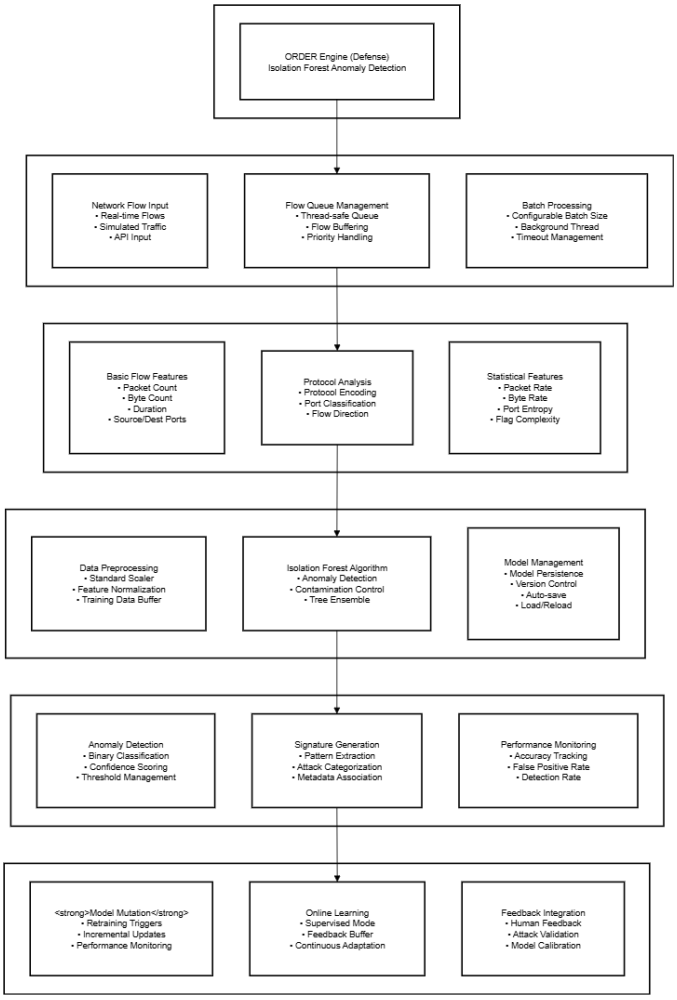

Fig. 2. ORDER module components and data flow

C. BALANCE Module: Behavioral Adaptation and Learning

The BALANCE module controls the evolutionary adaptation of the system, it iteratively refines both system parameters and architectural configurations based on feedback about performance from ORDER and CHAOS modules. It utilizes various algorithms coordinated in order to maximize system performance under a wide range of operating conditions.

The genetic algorithm part of the system retains a number of populations of systems, where each member is one set of parameters for the detection algorithms. Fitness evaluation takes into account various objectives such as detection rate, false positive rate, computational efficiency and adversarial robustness. Then, new configuration candidates are created by the selection, crossover and mutation operations which interact with the attacks using CHAOS.

The reinforcement learning part uses policy gradient methods to learn optimal real time decision making inside the ORDER module. It learns the best threshold values, algorithm selection policies and resource allocation strategies given current threat landscape features. The system balance exploration of new policies with exploitation existing ones using actor-

critic architectures.

The meta learning part enables fast adaptation to new environments and new types of attacks by learning generalizable adaptation policies. It keeps a library of successful adaptation experiences and transfers the learning for convergence speed-up in new deployment scenarios.

IV. Implementation

A. ORDER Module Implementation

The ORDER module is implemented using core machine learning algorithms from Python's scikit learn library, with custom optimization for high throughput network traffic processing. This is an ensemble of 200 trees and tuned contamination (0.05-0.15 chosen with grid search over different validation datasets).

Preprocessing of feature extraction deals with several data sources, such as pcap files, JSON logs or datasets stored in CSV. This work describes how the system leverages efficient memory management to deal with large-scale datasets, and provides incremental learning capabilities as well as data streaming pipelines. We solved the out of memory problem during mass dataset walking, by adopt data batch processing with sliding window and feature dimension reduction method such as PCA.

The Algorithm implementation is accelerated by optimized processes for real time computation. We developed a variant of the Isolation Forest algorithm to be amenable to incremental updates, so that new traffic patterns could be learned continuously without requiring retraining of the full model. Threading and multiprocessing versions are included to make multiplexing several traffic streams possible at the same time.

```
class OrderModule:
    def __init__(self, config):
        self.isolation_forest = IsolationForest(
            n_estimators=200,
            contamination=0.1,
            random_state=42
        )
        self.feature_extractor = NetworkFeatureExtractor()

    def detect_anomalies(self, traffic_batch):
        features = self.feature_extractor.extract(traffic_batch)
        anomaly_scores = self.isolation_forest.decision_function(features)
        return anomaly_scores < self.threshold
```

B. CHAOS Module Implementation

The CHAOS module is responsible for generating synthetic attack traffic used to evaluate the ORDER module during experimentation. Rather than relying on real adversarial machine learning methods, CHAOS operates using rule-based manipulation of known attack patterns from the KDD Cup 1999 dataset. Attack samples are selected from specific classes (DoS, Probe, R2L and U2R) and injected into the traffic stream at configurable rates to simulate changing threat conditions.

At its current stage, CHAOS does not implement gradient-based or GAN-driven adversarial example generation. Instead, it provides a controlled mechanism for replaying and remixing available attack traffic to stress-test detection models under varying proportions of benign and malicious inputs. This design allows the framework to explore the impact of shifting threat distributions without requiring external datasets or live packet captures.

Additional adversarial learning capabilities—such as FGSM/PGD perturbation or generative attack synthesis—remain planned extensions for future work.

C. BALANCE Module Implementation

The BALANCE module is intended to support automated adjustment of system parameters based on observed performance, but its current implementation remains at a preliminary stage. In the present version of KRONOS, BALANCE performs configuration management by allowing thresholds, model selections and dataset sampling ratios to be modified manually through a unified control interface.

Although the framework is structured to incorporate evolutionary search and reinforcement learning in future iterations, no genetic algorithm, policy optimisation or meta-learning methods were executed as part of the experimental results reported in this paper. Instead, BALANCE serves as a coordination layer that logs model behaviour and provides placeholders for future modules that will explore adaptive tuning strategies.

Future development includes integrating DEAP for evolutionary search and experimenting with reinforcement learning algorithms to automate parameter selection under dynamic threat conditions.

V. Evaluation Dataset

This work evaluates the KRONOS framework using the KDD Cup 1999 dataset, a widely used benchmark for intrusion detection research.

a) Dataset Summary: The processed dataset contained 49,096 samples, consisting of 41 features and five traffic categories: Normal, DoS, Probe, R2L and U2R. A significant class imbalance was present, with DoS and Normal traffic dominating the dataset (over 98% of samples), Probe traffic accounting for less than 1%, and R2L and U2R together representing under 0.1%. Experiments used an 80/20 stratified train–test split, and five-fold cross-validation was applied during training to ensure that evaluation metrics reflect generalisation performance despite skewed class distributions.

The dataset contains normal traffic and four attack categories (DoS, Probe, R2L and U2R). After preprocessing and feature encoding, 41 attributes were retained.

For all experiments, we used an 80/20 train–test split with stratified sampling to preserve class distribution, and applied five-fold cross-validation during training. Although UNSW-NB15 and NSL-KDD are targeted for future integration, they were not included in this evaluation due to preprocessing complexity and time constraints.

A. Core Datasets

1) UNSW-NB15: : The UNSW-NB15 dataset is an excellent option with a combination of real modern normal activities and synthetic contemporary attack behaviors. While old, the NSL-KDD and KDD Cup 1999 datasets are established benchmarks

for testing intrusion detection systems with a large variety of attack types that are still relevant to baseline against. Combined, these data sets offer a desired level of diversity in both traffic characteristics and threat vectors necessary for measuring the adaptivity of our approach.

B. Preprocessing and Feature Alignment

One important problem in utilizing multiple datasets is to ensure feature consistency for training and inference purpose of the model. The data characteristics and the feature sets of each data set are not the same. To mitigate these issues, we designed a single preprocessing pipeline which normalizes data and generates a shared feature representation for all datasets. This includes statistical normalization, the treatment of categorical features, and fine-grained alignment of semantically similar features such that its flexible training can combine the datasets while we can still evaluate on datasets independently to test for generalization. This method alleviates dataset-dependent biases, and benefits the generalization capability of model on unseen network environments.

VI. Results and Performance Evaluation

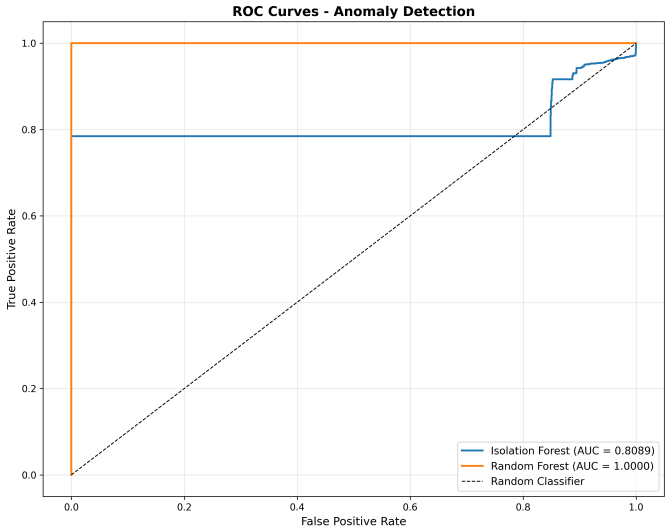

Fig. 3. ROC curves comparing anomaly detection models, including Isolation Forest and Random Forest baselines.

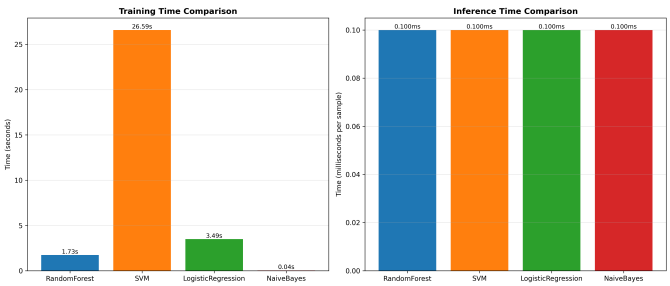

Fig. 4. Training and inference time comparison across classifiers.

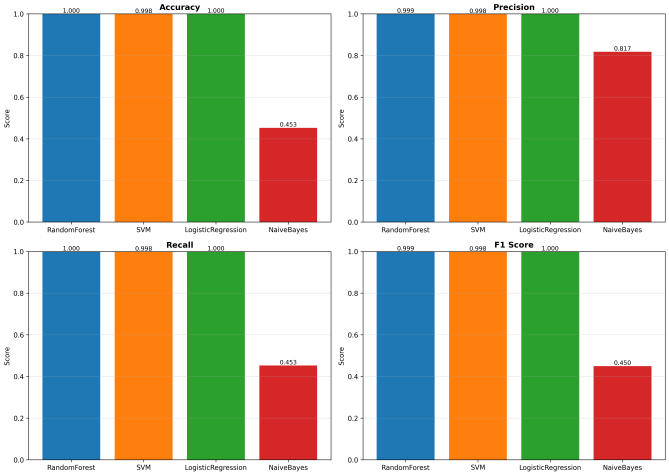

Fig. 5. Accuracy, precision, recall, and F1-score comparison across supervised classifiers.

A. Detection Performance

Experiments on the KDD Cup 1999 dataset show that classical machine learning models achieve strong detection capability across major attack categories. Random Forest, Logistic Regression and Support Vector Machine obtained accuracy and F1-scores above 99.8%, with low false positive rates. Naïve Bayes performed significantly worse (45.25% accuracy), and Isolation Forest showed limited utility for multiclass detection.

These results indicate that traditional supervised classifiers remain highly effective on structured intrusion detection data, while unsupervised anomaly detection methods require further tuning for balanced multiclass settings.

B. Runtime Performance

Training efficiency varied across models. Naïve Bayes and Isolation Forest trained in less than one second, Random Forest required approximately 1.7 seconds, and SVM was the most expensive at 26 seconds. Despite these differences, all models supported fast inference, averaging around 0.1 ms per sample, indicating that real-time detection workloads are feasible once models are trained.

C. Class-Level Behaviour and Limitations

Across all high-performing models, Normal, DoS and Probe traffic were detected with near-perfect precision and recall. Performance on minority classes was constrained by severe imbalance in the dataset. Only 12 R2L and a single U2R sample appeared in the test set, resulting in reduced recall for R2L and zero recall for U2R across models.

These findings highlight that while overall accuracy appears high, real-world deployment would require either data augmentation or rebalancing strategies to ensure equitable performance across rare attack types.

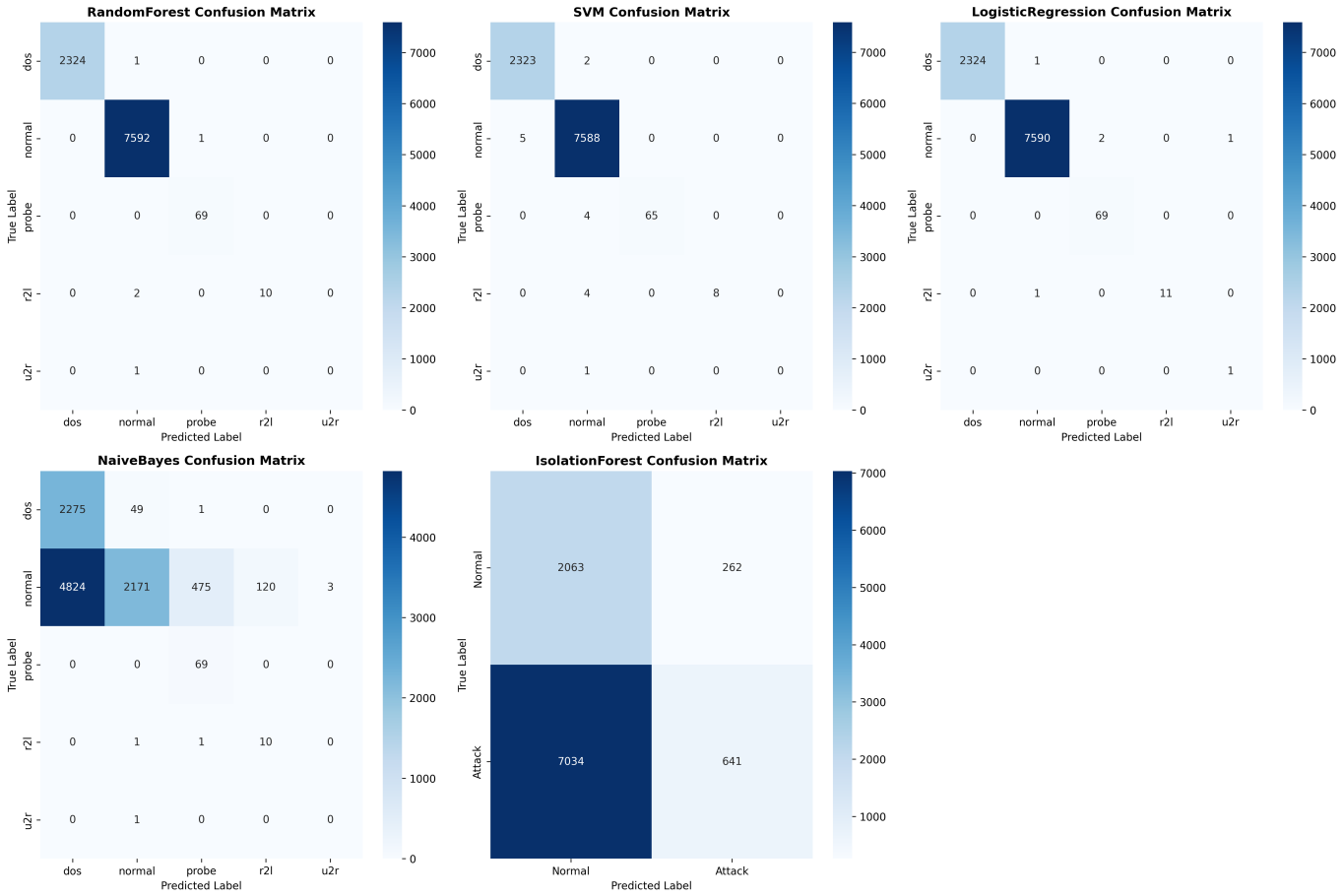

Fig. 6. Confusion matrices for Random Forest, SVM, Logistic Regression, Naïve Bayes, and Isolation Forest classifiers.

VII. Discussion

A. Limitations and Challenges

Although the overall results are promising, our approach has some limitations. The computational costs associated with frequent re configuration are significant, and the architectural complexity of the system is difficult to maintain. In addition diversity of fitness scenarios plays a key role to avoid convergence toward suboptimal configurations as well as to the overall performance. In addition, there is a compromise between adversarial robustness and benign traffic performance that needs to be calibrated well.

B. Real Time Deployment Considerations

Care should be taken in integrating the framework into wireline infrastructure for latency handling. Added to that is it's being an autonomous system, which requires strong validation frameworks to guarantee that security posture gets better over time rather than worse. That includes automated testing of new configurations and a safe rollback. Furthermore, rigorous audit trails record decisions made by automated algorithms in order to fulfill regulatory and compliance needs.

C. Future Work and Enhancements

There are a number of interesting research directions that arise from this work. Further down the road, future versions of KRONOS could leverage deeper merges between reinforcement learning (RL) and ORDER for quicker decision making process employing technologies such as DDQN for superior strategical planning. We will also incorporate live threat intelligence feeds that would allow our system to adapt proactively.

As a way to improve the abilities of cooperative defense, we will investigate federated learning that enables all deployments to privately share threat intelligence with each other. Lastly, you need advanced model management techniques to ensure continuous learning is dependable. This includes automatic retraining processes that weigh new insights against the risk of concept drift, and implementing versioning based on checkpoints for machine learning models to rapidly fix undesirable changes.

VIII. Conclusion

This paper introduced KRONOS, a modular framework designed to explore adaptive and learning-driven approaches to intrusion detection. The current implementation focuses

on the ORDER module and evaluates a range of classical machine learning models on the KDD Cup 1999 dataset. Experimental results show that models such as Random Forest, Logistic Regression and Support Vector Machine achieve high detection accuracy and low false positive rates, demonstrating the continued strengths of supervised learning methods in structured intrusion detection tasks.

While the framework incorporates placeholders for attack simulation (CHAOS) and automated parameter adjustment (BALANCE), these components remain at an early developmental stage and were not included in the empirical evaluation. Their integration represents a key next step toward enabling dynamic adaptation under changing threat conditions.

Future work will extend KRONOS beyond static dataset evaluation by incorporating additional datasets, implementing evolutionary search and reinforcement learning within the BALANCE module, and exploring synthetic or adversarial traffic generation to enhance robustness.

Acknowledgment

The authors would like to thank the creators of the KDD Cup 1999 dataset, as well as the teams behind NSL-KDD and UNSW-NB15 whose datasets were referenced during framework design. We also thank the open-source community for developing the machine learning libraries that enabled this research.

References

[1] M. H. Bhuyan, D. K. Bhattacharyya, and J. K. Kalita, "A survey on network-based intrusion detection systems," *Computer Science Review*, vol. 11-12, pp. 45–86, 2014.

[2] M. Roesch, "Snort: Lightweight intrusion detection for networks," in *Proceedings of the 13th USENIX conference on System Administration (LISA '99)*, 1999, pp. 229–238.

[3] S. Axelsson, "The base-rate fallacy and its implications for the difficulty of intrusion detection," in *Proceedings of the 6th ACM conference on Computer and communications security*, 2000, pp. 1–7.

[4] V. Paxson, "Bro: a system for detecting network intruders in real-time," *Computer Networks*, vol. 31, no. 23-24, pp. 2435–2463, 1999.

[5] OSSEC Project, "OSSEC: Host-based Intrusion Detection System," https://www.ossec.net/, 2019, accessed: 2024-05-17.

[6] A. L. Buczak and E. Guven, "A survey of data mining and machine learning methods for cyber security intrusion detection," *IEEE Communications Surveys & Tutorials*, vol. 18, no. 2, pp. 1153–1176, 2016.

[7] G. Kim, S. Lee, and S. Kim, "Machine learning for intrusion detection," in *2014 international conference on information and communication technology convergence (ICTC)*. IEEE, 2014, pp. 65–70.

[8] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *2008 Eighth IEEE International Conference on Data Mining*. IEEE, 2008, pp. 413–422.

[9] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, and R. C. Williamson, "Estimating the support of a high-dimensional distribution," *Neural Computation*, vol. 13, no. 7, pp. 1443–1471, 2001.

[10] B. Biggio and F. Roli, "Wild patterns: Ten years after the rise of adversarial examples," *Pattern Recognition*, vol. 84, pp. 317–331, 2018.

[11] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," in *International Conference on Learning Representations (ICLR)*, 2015.

[12] N. Papernot, P. McDaniel, S. Jha, M. Fredrikson, Z. B. Celik, and A. Swami, "The limitations of deep learning in adversarial settings," in *2016 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 2016, pp. 372–387.

[13] M. Ring, S. Wunderlich, D. Grüdl, D. Landes, and A. Hotho, "A survey of deep learning for cybersecurity," *arXiv preprint arXiv:1904.09138*, 2019.

[14] A. Abraham, C. Grosan, and V. Snasel, "Evolutionary computation in cybersecurity: a review," in *Human-Computer Systems Interaction: Backgrounds and Applications 2*. Springer, 2007, pp. 209–227.

[15] B. Aslahi-Shahri, R. Rahmani, M. Chizari, A. Maralani, M. Eslami, and M. Ghasemi, "A review on application of genetic algorithm in network intrusion detection," in *2013 7th International Conference on e-Commerce in Developing Countries: With focus on e-Security (ECDC)*. IEEE, 2013, pp. 1–6.

[16] F. Cuppens and A. Miège, "Managing alerts in a multi-intrusion detection environment," in *Proceedings of the 17th Annual Computer Security Applications Conference (ACSAC)*. IEEE, 2002, pp. 22–31.

[17] T. T. Nguyen and V. J. Reddi, "Deep reinforcement learning for cyber security: A comprehensive survey," *arXiv preprint arXiv:1905.13295*, 2019.